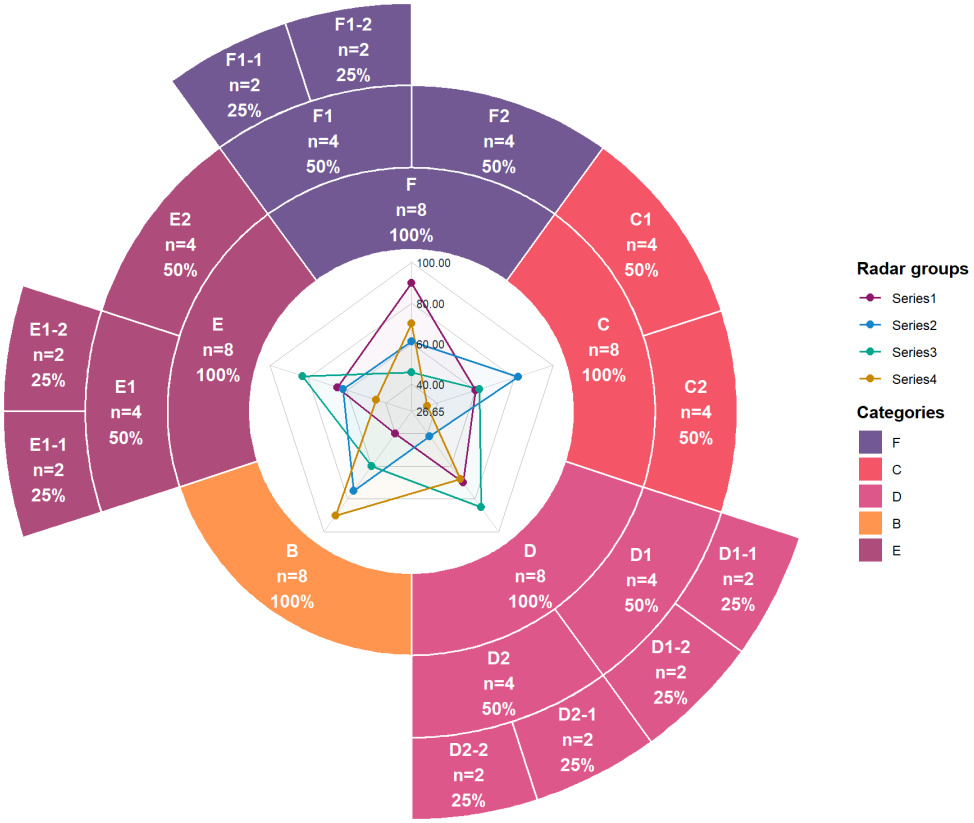
radarsunburst：看懂、试用和检查一本通

2026-10-20 阅读审查版。包版本 0.1.0，尚未提交 CRAN。

这次把 136 页的源码审查书改为围绕问题讲解的短版。你需要理解的是：输入什么、怎样计算、怎样变成图、如何亲手验证，以及以后怎样开发 R 包。完整源码直接保留在仓库，不再塞进阅读正文。

本书中的“已测试”指自动检查或 Codex 检查，不代表你本人已经看过。发布前最后一页由你逐项确认。旧版长手册另存为历史对照；当前接口以本书和 R 帮助为准。

阅读路线：先看第 1 到 11 节理解输入和用法；再照第 14 到 22 节安装与验收；想学开发时看第 12、24、25 节。第 24 节另有完整开发练习，第 26、27 节保留 skill 体验和改进建议。PDF 左侧书签可以直接跳转。



当前 geom_radsun() 生成的演示图。先读输入映射，再按步骤亲手验收。

1. 先认识输入：database 在这里就是一张表

不需要搭建数据库服务器。可以直接传入 R 的 data.frame，也可以先把 CSV 读成表。最容易准备的是“宽表”：一行一个条目，多个指标分开放在不同列。

team	category	subcategory	quality	efficiency	innovation
A	X	X1	70	50	60
A	X	X2	90	70	80
B	X	X1	40	60	30
B	Y	Y1	60	80	50
B	Y	空白	80	100	70

team 是雷达分组；category、subcategory 是大类和子类；后三列是三个数值指标。它们可以改成你自己的列名。

一行会给外圈贡献一次计数，也会把该行的指标值放入所属组的平均计算。外圈不按 team 拆成多张图，也不按指标值的大小决定扇区宽度。

最后一行的 subcategory 为空，意思是“这个条目到 Y 为止”。不是丢掉这一行，也不是自动编造一个“其他”类别。

2. 哪列对应哪根轴、哪条线、哪一圈

你指定的参数	对应输入列	图上发生什么
metrics	quality、efficiency、innovation	三根雷达轴；按给定顺序从正上方顺时针排列
group	team	A、B 两组各形成一条雷达轮廓
hierarchy	category、subcategory	从内到外两层旭日分类
percent	root 等选项	控制标签和悬停里的分母，不改变扇区宽度

用列名字符串告诉函数“去哪一列找数据”：

R 控制台

```
metrics <- c("quality", "efficiency", "innovation")
hierarchy <- c("category", "subcategory")
```

metrics 是多个列名；group 是一个列名；hierarchy 是按层级排列的多个列名。c() 的作用是把几个值放成一个向量，箭头 <- 的作用是给结果起名字，方便后面重复使用。

改变 metrics 的顺序会改变雷达轴的位置；改变 hierarchy 的顺序会改变分类树的含义，不能随便交换。group 与 category 不必一一对应：A、B 是一套分组，X、Y 是另一套分类。

默认已去掉整个灰色指标环，静态图不会自动写出每根雷达轴的名字。请在图注写明轴序；交互悬停仍显示指标名。需要轴名环时可显式开启，本书后面会演示。

3. aes() 在这里为什么不负责选指标

平时画散点图时，aes(x = quality, y = efficiency) 表示“一列当横轴，一列当纵轴”。这张复合图却没有这样一对共同的原始横纵轴：中心是多指标雷达，外圈是分类树。

因此，本包用 metrics、group、hierarchy 指定原始列；随后自行算出画多边形所需的 x/y。geom_radsun() 不继承父图的 aes() 映射。这是当前接口的明确约定，不是漏写了 aes()。

内部绘图用到的映射	真正来源	你怎样控制
x、y	包计算的平面坐标	指标顺序、范围、半径参数
fill	顶层分类 X、Y	root_colors 或后加 fill scale
colour	雷达组 A、B	group_colors 或后加 colour scale
group	多边形/曲线的内部编号	由包生成，和输入参数 group 不是同一个层面的东西
size	标签缩放后的字号	label_size、min_label_size 等参数
text	悬停文字	由计数、比例、组名和指标名生成

不需要在数据里准备 x、y、角度、百分比、颜色或悬停文字列。也不要用 aes(fill = team) 期待改变本组件的分组规则；请使用上表对应的参数或 scale。

geom_radsun() 没有独立的 mapping 参数，不支持用 aes() 重新定义其统计规则。你额外加的普通 geom_point() 仍可有自己的 aes()；它和本组件是各自的数据层。

4. 输入列需要满足什么条件

部分	要求	原因
表	至少一行；列名唯一且非空	否则无法知道取哪一列
metrics	至少三个不同的数值列；没有 NA、NaN、Inf	三个轴才构成雷达面；不能把缺测当作零
group	一列；不能缺失或空白	每行必须知道放进哪一组均值
hierarchy	至少一层；顶层不可为空	每行必须归到一个根类别
更低层级	可为空，但结束后不能再填下一层	防止出现“没有父类却有子类”的路径
自定义范围	两个递增的有限数，包住所有原始指标值	防止静默截断数据

分组和分类可以是字符、因子、数值编码或逻辑值；指标必须真的是 numeric，字符 "70" 不算数值 70。多余列会忽略，所以 id、name、sample_weight 都不是必填项。

R 控制台

```
d <- read.csv("你的数据.csv", fileEncoding = "UTF-8-BOM",
              stringsAsFactors = FALSE, check.names = FALSE)
names(d)       # 看列名是否与你填写的一样
str(d[metrics]) # 看指标是否真的读成数值
anyDuplicated(names(d)) # 0 表示没有重复列名
```

这不是自动清洗工具：不默认去重、不加权、不补零，也不自动把预算和评分换到同一单位。先确定一行代表什么，再决定是否去重和怎样转换单位。

5. 用五行数据亲手算一遍

下面是本书一直使用的小表。先在 R 控制台运行，后面所有试验都沿用它。

R 控制台

```
d <- data.frame(
  team = c("A", "A", "B", "B", "B"),
  category = c("X", "X", "X", "Y", "Y"),
  subcategory = c("X1", "X2", "X1", "Y1", ""),
  quality = c(70, 90, 40, 60, 80),
  efficiency = c(50, 70, 60, 80, 100),
  innovation = c(60, 80, 30, 50, 70)
)
metrics <- c("quality", "efficiency", "innovation")
hierarchy <- c("category", "subcategory")
```

A 组两行，质量均值 (70 + 90) / 2 = 80；效率 60；创新 70。B 组三行，三个均值分别是 60、80、50。因此两条雷达轮廓应当不同。

外圈数的是行：X 有 3 条，Y 有 2 条。X1 有 2 条，X2 有 1 条，Y1 有 1 条。根节点两个，子节点三个，一共五个节点。

为什么不数五个叶子？因为同一条分类路径会合并计数。为什么不只数四条？因为那条子类空白的 Y 仍然是有效条目。

6. 四种 percent 到底区别在哪

仍看 X1：它有 2 条，父类 X 有 3 条，全表有 5 条。这里仅有两层，所以 X1 的 parent 和 root 恰好相同；更深的树就可能不同。

选项	X 的标签比例	X1 的标签比例
none	不显示比例，只显示名称和数量	同左
parent	3 / 5 = 60%	2 / 3，约 66.67%
root	3 / 3 = 100%	2 / 3，约 66.67%
global	3 / 5 = 60%	2 / 5 = 40%

换比例只是在回答不同问题：“占父类多少”“占所属大类多少”“占全体多少”。原始数量不变，扇区也不变。默认 root，因此每个大类根节点都显示 100%，不能把各个根的 100% 相加。

三层例子：全表 40 条，D 有 8 条，D1 有 4 条，D1-1 有 2 条。D1-1 占父类为 50%，占根 D 为 25%，占全表为 5%。

这些比例都保存在 radar_sunburst_data() 的结果里。画图时不必把输入值换成百分比；手工先换成百分比再输入，会改变本来的数据含义。

不同父类下同名的子类会分别计数。默认顺序按数据里首次出现排列；root_order 和 group_order 可指定完整的大类/雷达组顺序；更细的 branch_order 用法查函数帮助，避免为了调图把分类含义改掉。

7. 雷达的长短、角度和外圈空白怎么来

雷达先找所有原始指标的最小值和最大值，再向两边各加跨度的 5%。五行小表是 30 到 100，跨度 70，所以默认范围是 26.5 到 103.5。先找原始极值、后算均值，可以避免平均之后把极端值藏起来。

某个值离中心多远，取决于它在这个范围内的位置。通俗写法是“值减下界，再除以整个跨度”。所以中心标的是 26.5，不是强行标 0。若你希望中心是 0、外边是 100，就设 radar_limits = c(0, 100)。

所有指标共用一个范围；改变样本子集会改变自动范围。比较两张不同子集的图时，最好明确指定相同范围，否则视觉差异可能来自刻度而非数据。

负数和零可以使用。全部原始指标都一样时，跨度为零，包会在两侧各扩展 max(1, 该值绝对值的 5%)，避免除以零。例如全部为 0 时得到 -1 到 1。

外圈把整圆按数量分份。X 占五行里的三行，所以占整圆 60%；Y 占 40%。子类再分父扇区。Y 有两行但 Y1 只有一行，因此 Y 的下一层一半画 Y1，另一半留白。

这个空白是在诚实表达“这一行没有更细分类”，不是漏画。将它分给 Y1 会误导读者以为两行都有 Y1。

指标角度等分，分类角度按计数。两者没有默认的一一对应关系：某条雷达轴正好朝着某个分类扇区，不表示该指标专属于那个类别。

8. 两个绘图入口，现在怎样分工

你想要的加号写法

R 控制台

```
library(radarsunburst)
library(ggplot2)
p <- ggplot(d) +
  geom_radsun(metrics = metrics, group = "team",
              hierarchy = hierarchy) +
  theme_radar_sunburst()
p
```

geom_radsun() 把雷达、外圈和标签等组件放进 ggplot；theme_radar_sunburst() 负责背景、标题和图例的默认外观。固定圆形比例和默认颜色刻度已经随组件加入，不需要再手动补 coord_fixed()。

一键写法仍然保留

R 控制台

```
quick <- radar_sunburst(d, metrics, "team", hierarchy)
quick
```

这两段代码使用同一套计算与绘图规则；一键函数相当于帮你把 ggplot()、geom_radsun() 和默认主题接起来。它们不会各维护一套不同的均值和比例算法。

目前公开函数总数是五个：统计、复合组件、一键图、主题、交互转换。内部辅助函数留在包里，不需要用户记住。

9. 数据继承和 aes 的实际边界

R 控制台

```
# 从 ggplot(d) 取表：
p1 <- ggplot(d) +
  geom_radsun(metrics = metrics, group = "team",
              hierarchy = hierarchy)
# 直接在组件里指定表：
p2 <- ggplot() +
  geom_radsun(data = d, metrics = metrics, group = "team",
              hierarchy = hierarchy)
```

第一种更接近常见 ggplot 写法；第二种适合你想明确写出本组件的数据来源。两处都写 data 时，以 geom_radsun(data = ...) 为准。两处都不提供表，会报错。

如果父图写了 aes(x = quality, y = efficiency)，它不会变成本组件的雷达轴规则。本组件只继承表，自己生成坐标；这种做法也避免了汇总之后再去查不存在的原始列。

已有的标题、主题和其他图层会保留。但所有图层仍共享同一张图的坐标和颜色刻度：不要以为加入复合组件后，原有散点图还能继续使用独立的一套横纵轴。

当前每张图支持一个 geom_radsun()。不承诺分面或对数坐标；这些会涉及“每个面板如何单独汇总”和“圆形怎样变换”等额外设计。重复加第二个组件会明确报错。

想加注释可以，但位置采用计算后的平面坐标；annotate(x = 80) 并不表示某根雷达轴上 80 分的位置。

10. 换主题、换颜色和处理指标名称

R 控制台

```
p + theme_minimal() + labs(title = "五个条目的比较")
p + theme(legend.position = "bottom")
p + scale_colour_manual(values = c(A = "#0072B2", B = "#D55E00"))
p + scale_fill_manual(values = c(X = "#4477AA", Y = "#AA3377"))
```

colour 改雷达组，fill 改大类。新 scale 放在组件后；ggplot 提示“替换已有 scale”是正常消息，因为它确实在替换默认配色。雷达半透明填充跟随组线色，不占用外圈的大类填色规则。

主题控制标题、图例、背景；扇区内部文字属于绘制内容，要用 label_size、label_colour 等参数改。大幅改字体或图幅后需要肉眼检查，小扇区标签可能缩小后仍放不下，因而被隐藏，但数量和悬停信息仍保留。

R 控制台

```
named <- radar_sunburst(d, metrics, "team", hierarchy,
  metric_ring = c(1.06, 1.55), show_metric_labels = TRUE,
  metric_labels = c("Quality", "Efficiency", "Innovation"))
```

这段只是演示显式恢复指标环的功能，不会改变默认图。你的默认要求仍是整个灰环去掉。静态导出时如果不恢复指标环，应在图注写清“从上方顺时针：质量、效率、创新”。

中文显示依赖电脑字体和绘图设备。包不会安装字体。默认文字横排更利于 Plotly 转换；旋转标签不保证交互图保持同样角度。

11. 保存图片和转换交互图

R 控制台

```
ggsave("my-chart.png", p, width = 10, height = 8, dpi = 300)
direct <- plotly::ggplotly(p, tooltip = "text")
w <- radar_sunburstly(p, width = 900, height = 700)
w
htmlwidgets::saveWidget(w, "my-chart.html", selfcontained = FALSE)
```

ggsave() 写图片；ggplotly() 把 ggplot 的图层转为浏览器能画的交互元素；radar_sunburstly() 在这个转换上继续整理悬停、圆形比例和雷达图例联动。

转换后的对象属于 Plotly，不再用 ggplot 的加号添加主题；先把 p 改好，再转换。交互通常包括悬停、缩放、组显隐，不包括点击扇区钻取下一层，也不自动筛选原始数据库。

保存 HTML 时，selfcontained = FALSE 会生成 HTML 和相邻资源目录，二者要一起保留、一起分享。只发 HTML，别人可能看到空白。用 TRUE 可以打包成一个文件，但需要作者电脑有 Pandoc。

直接 ggplotly() 能显示交互；推荐用 radar_sunburstly() 做最终浏览器验收，它额外让同一雷达组的线、点和填充一起显隐，并让装饰层不打扰悬停。

“成功创建 widget”只说明对象构造成功；真正看得到字、悬停是否正确、缩放后是否仍为圆形，需要实际打开 HTML 检查。

12. 项目架构：看这一条路线就够

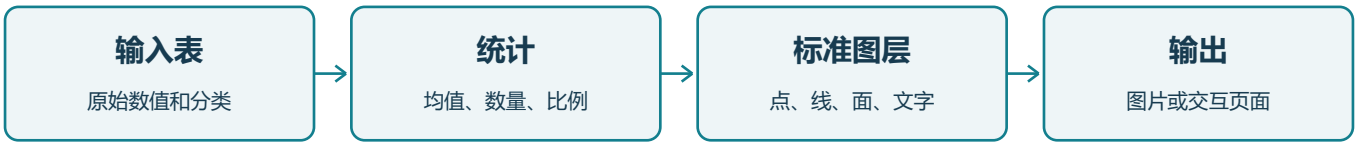
原始表 -> 校验与统计 -> 坐标与标准图层 -> ggplot -> 图片或 Plotly。

文件	主要工作	可以怎样理解
R/data.R	检查数据、求均值、数分类、算比例与角度	先算对答案
R/geom-radsun.R	接收加号组件，找数据，再调用共同绘图核心	接线员
R/plot.R	根据统计结果生成雷达和扇区；一键入口也在这里	把答案画出来
R/theme.R	默认主题	决定外观环境
R/interactive.R	把图转为并整理 Plotly 交互	浏览器适配
R/demo-data.R	说明演示数据的字段和来源	数据说明书

为什么先统计、再画图？这样可以独立检查数量是否正确。如果图片看起来漂亮但计数错误，直接看统计表就能发现，不必从像素反推。

为什么不用全新的底层 Geom？因为本图可以由标准多边形、线、点、文字组合而成，既容易复用 ggplot 功能，也便于 Plotly 识别。geom_radsun() 是一个组合组件；名字里有 geom，并不强迫它只能产生单一图层。

加号能识别它，是因为包登记了对应的添加方法。你不需要手动调用这个内部方法；只需知道“组件不是整张图，把它加到 ggplot 后才形成可绘制结果”。



13. 文件夹里哪些值得你看

文件或目录	你需要知道的事
DESCRIPTION	包名、版本、真实作者、许可和依赖，相当于身份证
NAMESPACE	哪些函数可公开调用、哪些方法已登记，由文档工具生成
R/	真正运行的包代码，按上一页的五个职责读
man/	R 里的帮助页，由代码旁的说明生成
tests/	预先写好的算例和错误输入检查
vignettes/	可执行的英文完整教程
data/、inst/extdata/	可用 data() 加载的演示表，以及可查看的 CSV
data-raw/	开发时把 CSV 变成包内数据的脚本
README.md、NEWS.md	快速入口、用户可见的变化
.github/	在不同系统自动构建和检查
tools/	开发、手动演练和检查工具，不随 CRAN 包安装
docs/	中文说明、架构、审查与维护记录，不随 CRAN 包安装
artifacts/	图、源码包和检查日志，不是运行代码
.Rbuildignore	告诉构建工具哪些开发文件不放进源码包

原来的 radar_sunburst.Rmd 和 render.R 留作原型对照，不是新包的运行入口。tmp/、output/ 是临时或导出文件；第三方工具缓存不属于本包实现。不要因为它们在文件夹里，就把它们全部上传为 R 包。

14. 手动试用之前：先把包安装到本地

如果只是试用现成源码包，不需要自己写函数或生成帮助。先用与你日常工作一致的 R/RStudio；下面的代码在 R 控制台运行。

R 控制台

```
# 只在你选择安装依赖时运行，包不会自动替你安装：
install.packages(c("ggplot2", "rlang"))
# 想试交互再安装：
install.packages(c("plotly", "htmlwidgets"))

lib <- "E:/R-libraries/radarsunburst-review"
dir.create(lib, recursive = TRUE, showWarnings = FALSE)
.libPaths(c(lib, .libPaths()))
install.packages(
  "E:/desktop/radarsunburst/artifacts/release/radarsunburst_0.1.0.tar.gz",
  repos = NULL, type = "source", lib = lib
)
```

上面建的是你自己选择的测试包库。repos = NULL 表示从本地文件装，不去 CRAN 找同名包。type = source 表示源码包。这个包本身没有需要编译的 C/C++ 代码。

装好后重启 R 会话，再设置 .libPaths() 并加载，防止仍在使用旧版本或开发环境中 load_all() 的对象。安装是把包放到本机的包库里；不等于提交到 CRAN，也不等于发布 GitHub release。

请确认这个文件是本次 geom_radsun 版本，而不是历史归档中的旧包。包仍是尚未发表的 0.1.0，所以只看版本号不足以判断是否装对。

15. 如果要亲手从源文件打包

“构建”把文件夹整理成一个 tar.gz；“安装”把这个 tar.gz 放到包库；“加载”才是本次 R 会话开始使用它。三个步骤不要混为一谈。

下面在 PowerShell 终端运行，不要粘到 R 控制台。此电脑的路径仅作示例：

PowerShell 终端

```
Set-Location "E:/desktop"
& "D:/Rstudio&R/R-4.4.3/bin/x64/R.exe" CMD build "E:/desktop/radarsunburst"
```

成功后终端当前目录会有 radarsunburst_0.1.0.tar.gz。如果构建教程时报缺 knitr/rmarkdown/Pandoc，应先补开发工具；不要把失败当成已构建成功。

然后回到 R，按上一页安装步骤，把文件路径改成你刚构建的 tar.gz。你自己构建的包可能与最终发布制品具有不同哈希，因此不能挪用别人的检查报告。

开发者修改代码旁的帮助说明后，还要先运行 roxygen2::roxygenise("E:/desktop/radarsunburst") 更新 man/ 和 NAMESPACE，再构建。普通试用者安装现成源码包时不需要运行 roxygen2。

开发检查还需要 testthat 等 Suggests 中的依赖；完整 CRAN 检查会检查教程和 PDF 手册，因而需要额外工具。这些是开发环境的要求，不是每位普通画图用户都需要装的东西。

完整检查的终端命令是 R CMD check --as-cran 后接具体 tar.gz 路径。它检查的是那一份文件，不是你后来继续修改的文件夹。

16. 在新会话里确认装对了

R 控制台

```
lib <- "E:/R-libraries/radarsunburst-review"
.libPaths(c(lib, .libPaths()))
library(radarsunburst)
library(ggplot2)
find.package("radarsunburst")
packageVersion("radarsunburst")
getNamespaceExports("radarsunburst")
sessionInfo()
```

应看到你选的包库路径，以及五个公开函数：geom_radsun、radar_sunburst、radar_sunburst_data、theme_radar_sunburst、radar_sunburstly。内部添加方法不应成为第六个公开函数。

这项检查的目的：排除“以为装了新包，实际还在加载另一个目录里的旧包”。也可以直接运行？geom_radsun 查看帮助；找不到该函数，通常说明没有安装/加载本次版本。

接着重新运行第 5 节的小表和两个向量定义。不要依赖上次会话残留的 d、metrics 等对象。若它们不存在，R 报 object not found 并不是包损坏，而是没有准备本次数据。

本包示例也可这样查看：data(radar_sunburst_demo)；dim(radar_sunburst_demo) 应得到 40 行、18 列。str() 可以帮助你检查哪些列是数值、哪些列是分类。样例数据是虚构数据，不应当作真实科研结论。

17. 手动检查一：先核对统计答案

R 控制台

```
s <- radar_sunburst_data(d, metrics, "team", hierarchy)
s$radar
s$radar_limits
s$nodes[, c("label", "count", "parent_proportion",
            "root_proportion", "global_proportion")]
```

预期 A 组为 80、60、70；B 组为 60、80、50；自动范围为 26.5、103.5。根 X 数量 3，Y 数量 2；X1 为 2，X2、Y1 各为 1；一共五个节点。

预期 X1 的 global_proportion 是 0.4，root_proportion 是 2/3。统计表保存 0 到 1 的比例，图上才乘 100 写成百分数；0.4 不是 0.4%。

检查目的：先确认答案算对，再看图。你可以用计算器复算 A 的质量均值、根分类行数和 X1 的分母，不需要理解所有内部函数。

再试原始 40 行演示表：四组均值可见下方；五个根各 8 条，共 21 个分类节点。指标顺序要与给定的五个名称一致。

组	稳定性	效率	创新	质量	协作
Series1	90	60	70	40	65
Series2	61	82	42	75	62
Series3	46	62	85	60	83
Series4	70	35	68	90	45

18. 手动检查二：两个入口和主题

R 控制台

```
p <- ggplot(d) +
  geom_radsun(metrics = metrics, group = "team",
              hierarchy = hierarchy) + theme_radar_sunburst()
quick <- radar_sunburst(d, metrics, "team", hierarchy)
p
quick
identical(attr(p, "radar_sunburst_summary"),
          attr(quick, "radar_sunburst_summary"))
p + theme_minimal() + labs(title = "我正在检查")
p + theme_dark() + theme(legend.position = "bottom")
```

预期两种入口的统计表相同，最后的 identical() 返回 TRUE。默认没有灰色指标环。主题切换后，标题/背景/图例应该改变，分类数量和雷达位置不应改变。

检查目的：确认新增的 geom 不是另外一套算法，也确认主题修改只是外观调整。普通 theme_minimal() 可能重新显示平面坐标轴和网格，这是主题本身的行为；想保留干净圆形外观，可以用本包默认主题再局部调整。

手工比较时请保持窗口/导出尺寸相同。图幅不同会影响标签适配，因此不能仅凭字的大小判断数据发生变化。若静态图上没有轴名，应核对图注的指标顺序，避免别人误读。

19. 手动检查三：比例、空白和固定范围

R 控制台

```
for (scope in c("none", "parent", "root", "global")) {
  print(ggplot(d) +
    geom_radsun(metrics = metrics, group = "team",
                hierarchy = hierarchy, percent = scope) +
    theme_radar_sunburst() + labs(title = scope))
}
fixed <- radar_sunburst(d, metrics, "team", hierarchy,
                        radar_limits = c(0, 100))
fixed
```

检查	预期	为了发现什么
切换 percent	扇区位置不变，只改标签内容	防止显示口径意外改变计数
root 根标签	X、Y 都是 100%	确认你接受这种分母含义
global 根标签	X 为 60%，Y 为 40%	确认全体占比可复算
Y 的下一层	只有一半有 Y1，另一半留白	防止把终止分支错误分给子类
固定范围	中心变为 0，最大界限为 100	确认雷达长短随指定刻度变化

雷达轮廓在切换固定范围时可以变，这是刻度变化的正常结果；原始均值不应变。切换百分比时雷达不应变，因为这个参数只管外圈的显示口径。

20. 手动检查四：故意输错，看报错是否有用

下面每次单独运行一项；出现预期错误后再继续，不要把整段错误测试当作一条必须全成功的流水线。

R 控制台

```
bad <- d
bad$quality[2] <- NA_real_
radar_sunburst_data(bad, metrics, "team", hierarchy)
radar_sunburst_data(d, c("quality", "missing", "innovation"),
                    "team", hierarchy)
radar_sunburst(d, metrics, "team", hierarchy,
               radar_limits = c(50, 90))
```

第一项应指向 quality 的第 2 行缺失；第二项应说缺少 missing 列；第三项应报告原始值超出范围，不能悄悄裁掉 30 或 100。

再试把 quality 变成字符、填 Inf、让 team 为空，均应被拒绝。三层分类跳级示例：

R 控制台

```
bad <- d
bad$detail <- "leaf"
bad$subcategory[5] <- ""
radar_sunburst_data(bad, metrics, "team",
                    c("category", "subcategory", "detail"))
```

第 5 行在没有子类时又有 detail，应提示跳级。检查目的不是“让程序永远不报错”，而是确认不合适的数据会及时被指出，避免生成看起来合理的错误图。

21. 手动检查五：真的打开交互页面

R 控制台

```
direct <- plotly::ggplotly(p, tooltip = "text")
direct
w <- radar_sunburstly(p, width = 900, height = 700)
w
htmlwidgets::saveWidget(w, "manual-chart.html",
                        selfcontained = FALSE)
```

操作	预期	检查目的
悬停 A 的质量点	A、quality、均值 80	防止指标名与轴序错配
悬停 X1 扇区	完整路径 X / X1、数量 2；root 分母 3	防止同名节点或分母出错
点击 A 图例	A 的线、点、填充一起隐藏，再点恢复	防止只隐藏其中一层
拖拽缩放，再重置	数量不变、圆形比例保持	防止交互后圆被拉扁
悬停文字/网格附近	配置转换器尽量排除装饰悬停	防止短文字遮住扇区信息
浏览器打开保存的 HTML	图可见、交互可用	防止只在 RStudio 中正常

配置转换器中的分类图例用于解释，不负责隐藏扇区。direct 是直接转换对照；最终组显隐和装饰悬停的验收以 w 为准。

截图看不出点击能否正常恢复。请真的操作一遍，并保留浏览器截图或写下失败步骤。不要把“文件保存成功”当成全部交互功能都通过。

22. 手动检查六：导出、中文和小扇区

R 控制台

```
ggsave("manual-chart.png", p, width = 10, height = 8, dpi = 300)
ggsave("manual-chart-small.png", p, width = 6, height = 5, dpi = 150)
```

打开实际图片检查边界、图例、刻度和文字，不要只看 RStudio 缩略图。缩小图幅后标签可能更难读；默认算法按典型图幅估算，不承诺任意尺寸都完美。

把 category 改成中文或较长名称，重画一次，检查是否乱码、被截断或和其他文字撞在一起。中文出问题时先确认字体与设备，不要把文字转成问号再说“兼容”。

做一份 100 行的表：一个大类下，85 行同一个子类，另外 15 行各是一个不同子类。小扇区可能不显示文字，但应仍有扇区，交互悬停应仍能看到完整信息。检查目的：确认隐藏的是放不下的标签，而非把少数类别从统计里删掉。

最后重启一次 R，从其他目录加载已安装包，只用显式读取的表完成绘图。检查目的：排除包暗中依赖项目目录、旧对象或 source() 过的文件。

完整演练也可在 R 中显式 source("E:/desktop/radarsunburst/tools/manual-acceptance.R")。它输出数值、图和临时 HTML，覆盖全部五个公开函数；文件在 tempdir() 下，会话结束前请复制需要保留的结果。它不会替你完成肉眼验收。

23. 自动测试能替你做什么，不能做什么

自动测试是在告诉电脑：“这个小例子应该得到这个答案，改完代码再算一遍。”它很适合反复检查计数、均值、输入错误和两个入口是否一致。

本次增加了组件测试：继承父图数据、显式数据优先、不污染父图映射、保留已有标题和主题、重复使用同一个组件规格，以及两种交互转换。当前自动套件在开发 ggplot2 和最低支持的 3.5.0 上各有 216 项断言通过。

216 是断言数，不是 216 个完全独立场景，更不是正确性的数学证明。后续增加测试后这个数字也会变。测试的重点是“确实验证行为”，而非把内部每一行原样再写一遍。

DESCRIPTION 声明 R 至少为 4.1，但本次没有单独在 R 4.1 上运行；不能把 ggplot2 的最低版本测试误读成所有旧 R 都已验证。实际运行的 R 版本与平台看发布检查记录。

自动测试可以确认“生成的悬停文字包含 80”，却不能充分证明“你在浏览器里点那个位置就真的看到 80”。它也不能替你判断轴序是否容易理解、字号是否舒适，或科学解释是否成立。

因此分三层：先看数字；再看静态图；最后操作交互图。R CMD check 是包的整体体检，还会检查文档和例子是否能运行，但不等于 CRAN 保证收录，也不等于维护者已经人工批准。

本书第 17 到 22 节就是人工补上的这部分。请写“未检查”“通过”或“失败 + 复现步骤”，不要把没操作过的项目直接勾通过。

24. 从这次项目学会开发下一个 R 包

先把问题说清楚：输入是什么、输出是什么、哪些值要拒绝。比如一个只求均值的小包，也先决定缺失值到底报错还是忽略，不能写到一半再默默决定。

开发工具是你主动安装的工具，不是未来用户必须安装的运行依赖：

R 控制台

```
install.packages(c("usethis", "roxygen2", "testthat", "devtools"))
usethis::create_package("E:/practice/profilekit", open = FALSE)
```

先创建存在的父目录 E:/practice，再执行上面操作。然后在 RStudio 打开新项目的 .Rproj；不要留在 radarsunburst 项目里运行“给当前项目加测试”的命令。

在新项目的 R/mean-profile.R 保存下面的函数。#' 注释给帮助工具读取；@export 表示公开；后面的 function 才执行计算。

R 控制台

```
#' Average a complete numeric profile
#' @param x A nonempty finite numeric vector.
#' @returns One number.
#' @export
#' @examples
#' mean_profile(c(2, 4, 6))
mean_profile <- function(x) {
  if (!is.numeric(x) || is.complex(x) || !is.null(dim(x)) ||
      !length(x) || any(!is.finite(x)))
    stop("Supply a nonempty finite numeric vector.")
  mean(x)
}
```

接下来的三页把帮助、测试、构建与安装接起来。练习请留在 profilekit，别把它的文件加进 radarsunburst。这里演示的是一个小而完整的包，不会真的替你上传。

24.1 开发练习：身份证、许可与测试

先打开新项目中的 DESCRIPTION。把 Title 改成准确的英文标题，Version 明确改为 0.0.1，Authors@R 填自己的真实姓名、邮箱和角色，Description 用几句英文解释功能。字段续行需要缩进，不要把它当成随意排版的普通笔记。

在这个新项目的 R 控制台运行：

R 控制台

```
usethis::use_mit_license()
usethis::use_testthat(edition = 3)
```

第一项写入许可文件和对应元数据；第二项准备测试框架。然后在 tests/testthat/test-mean-profile.R 保存：

R 控制台

```
test_that("valid inputs have the expected mean", {
  expect_equal(mean_profile(c(2, 4, 6)), 4)
  expect_equal(mean_profile(c(-2, 0, 2)), 0)
})
test_that("incomplete or wrongly shaped inputs are rejected", {
  expect_error(mean_profile(numeric()))
  expect_error(mean_profile(c(1, NA_real_)))
  expect_error(mean_profile(c(1, Inf)))
  expect_error(mean_profile("2"))
  expect_error(mean_profile(matrix(1:4, nrow = 2)))
})
```

前两项把数学答案写清；后五项保护输入契约。expect_error() 表示“这里本来就应该报错”，因此它通过时不表示函数接受了错误数据。先写有用的边界，不必追求每行代码都对应一条测试。

24.2 开发练习：生成帮助、测试与构建

在 profilekit 项目的 R 控制台运行：

R 控制台

```
devtools::document()
testthat::test_local()
pkgload::load_all()
mean_profile(c(2, 4, 6))
```

document() 读取 #' 注释，生成 man/mean_profile.Rd 和 NAMESPACE；test_local() 检查刚才保存的七项断言；load_all() 让你在开发时快速试用源文件。最后一行应得到 4。

不要手改生成出来的帮助页来掩盖代码与文档冲突。正确做法是修改函数上方的说明，再 document()。@param 说明输入，@returns 说明输出，@examples 提供可运行例子，@export 决定用户能直接调用的函数。

开发试用通过后，在 PowerShell 终端构建：

PowerShell 终端

```
Set-Location "E:/practice"
& "D:/Rstudio&R/R-4.4.3/bin/x64/R.exe" CMD build "E:/practice/profilekit"
& "D:/Rstudio&R/R-4.4.3/bin/x64/R.exe" CMD check --as-cran "E:/practice/profilekit_0.0.1.tar.gz"
```

先确认 build 成功再运行 check。check 的结尾要看 ERROR、WARNING、NOTE 各有多少；看到 DONE 不代表没有问题。“首次提交”的 NOTE 也应记录解释，不能假装不存在。

当前 radarsunburst 就曾在新增组件时出现方法签名 WARNING：图能画、单元测试也通过，但完整检查发现它和新 ggplot2 的通用接口参数不一致。修正后必须重建、重查，而不是只更新报告里的数字。

24.3 开发练习：安装后再验一次

重启 R；这一次别用 load_all()，直接安装刚构建的文件：

R 控制台

```
lib <- "E:/R-libraries/profilekit-review"
dir.create(lib, recursive = TRUE, showWarnings = FALSE)
.libPaths(c(lib, .libPaths()))
install.packages("E:/practice/profilekit_0.0.1.tar.gz",
                 repos = NULL, type = "source", lib = lib)
library(profilekit)
find.package("profilekit")
mean_profile(c(2, 4, 6))
?mean_profile
```

应从指定包库加载，均值应为 4，帮助中能看到输入、输出和例子。这个步骤发现的是“源文件开发时能跑，真正安装后却找不到函数/资源/依赖”的问题。

以后改函数时，重复这一圈：改规则与代码、改测试和说明、生成帮助、运行测试、构建、安装、检查。改变用户可见行为要写 NEWS；已发布版本之后的变化还需要合适的版本号。

依赖分两类：用户每次调用都需要的包放 Imports；只用于测试、教程或可选交互的包通常放 Suggests。可选功能在缺包时应给安装提示，不能在函数里偷偷 install.packages()。

本练习的实际函数只调用基础 R，所以不需要把 devtools、roxygen2 当成运行依赖塞进 Imports。开发工具帮助你造包，和用户运行包所需的工具是两回事。

24.4 怎样把下一步清楚交给 Codex

与其只说“帮我优化”，可以给出输入、输出、边界、希望保留的行为与验收方式。例如：

“在现有包里增加加号式 geom 入口，原一键函数继续可用。两者必须共用统计和绘图核心，保留无灰环默认图。参数用列名字符串；明确说明是否继承 data 和 aes。更新帮助、README、教程、NEWS。比较两种入口的结果，测试最低 ggplot2 与交互，并重新构建检查；暂不上传 CRAN。”

让每阶段留下可核对结果：接口签名和例子、已运行的测试、静态图、可打开的 HTML、安装后新会话结果、精确 tar.gz 与哈希。不是每一步都要停下来问你；真正影响表达含义或发布的决定才需要你审查。

发现问题时，用“输入 + 操作 + 实际结果 + 预期结果”反馈。例如：“输入五行小表，点击 A 图例，点消失但线还在；希望整组一起显隐。”这比“交互怪怪的”更容易产生准确的回归测试。

让 Codex 帮你写代码，不会转移维护者责任：指标单位是否合理、图形是否误导、作者与许可是否准确、维护邮箱能否持续收信，仍需要你作最终判断。

不要因为 GitHub 已同步，就把它理解为正式发布。你可以反复本地安装和审查；真正上传 CRAN 应以你最终确认的那一份制品为准。

25. 改代码之后，文档和 GitHub 怎样跟上

一次完整修改的顺序：说明预期行为 -> 修改 R/ -> 增加有意义的测试 -> 更新函数帮助 -> 更新 README/教程/NEWS -> 构建安装 -> 新会话和图形验收 -> 完整检查 -> 记录制品哈希。

README 负责“最快上手”；man/ 负责“参数和返回值准确”；vignettes/ 负责“完整工作流”；NEWS 负责“改了什么”。本书帮助你理解和审查，不替代机器可执行的帮助和测试。

Git 提交记录一组明确变更；push 把它同步到 GitHub。当前仓库是 diamondten8/radarsunburst，默认分支 master。把代码推上去，不代表上传 CRAN，也不代表已经创建正式 GitHub release。

GitHub Actions 会构建一份共享源码包，让 Windows、Linux、macOS 和 R-devel 检查同一份文件。应看每个平台的实际日志，不能只看绿色徽章。

外部工具故障和包代码错误要区分，但两者都不能写成“已经检查通过”。例如缺 TeX 会挡住 PDF 手册检查，应补工具再跑；不能偷偷跳过手册后说完整检查通过。

本次新增 geom 改变了源码，因此旧包的六份检查只是历史证据。当前制品、哈希和检查结果以 RELEASE-STATUS.md 为准。之后你提出任何影响包内容的修改，都要重新构建和检查。

26. skill 的实际体验：它是工作说明书

这里的 skill 是给 Codex 的工作说明和辅助工具，不会被安装进 radarsunburst，也不是用户画图时的运行依赖。r-package-engineer 的作用是提醒代理按 R 包的规则做事。

确实有帮助的部分：区分公开函数与内部辅助；禁止运行时自动装包或改工作目录；帮助、测试与接口同步；对真正构建出的 tar.gz 留下完整检查和哈希。它让“代码能跑”和“可准备发布”之间的差别更明确。

本次增加 geom 时，skill 的阶段规则也提醒我们：接口变了，相关测试、文档和旧发布检查都要更新。无需重建整个项目，但不能挪用旧候选包的检查结果。

这是单个项目的使用体验，没有做有无 skill 的对照实验，不能据此声称节约了多少时间或提高了多少成功率。

改进建议	为什么值得做
先检查 R、Pandoc、TeX、Python 依赖	缺工具应早发现，别到最后才失败
支持检查已有 tar.gz	跨平台必须复查同一制品，不能每次重建
检查摘要格式统一	让哈希、平台、ERROR/WARNING/NOTE 容易核对
区分“加强检查”和“跳过检查”的环境变量	不能把启用检查也误判成绕过

27. skill 怎样变得更好用

再加一个按需读取的可视化包说明页：输入列如何映射、组合组件与完整图的职责、ggplot2 版本兼容、Plotly 的实际浏览器验收。普通数值包不需要背负这些检查；涉及图形时再读取。

这次用户真正关心的事情还应更早进入设计：期待 ggplot + geom 的用法、要理解哪些参数、怎样亲手验收。只满足 R CMD check，而不确认用户的操作习惯，会让接口与教程来回返工。

给“完成”的定义补上证据层次：数字测试、图层结构、静态图、真实浏览器、维护者人工确认，分别记录。代理检查过不等于用户确认过；成功创建 HTML 不等于交互可用。

写 skill 时，用短主文件说明决策和边界，把详细步骤放到引用文件，把重复机械操作交给脚本。不要为了覆盖所有情况，把所有知识都塞进每次必读的主文件。

改 skill 后也需要试验：给它一个新包、一个现有包的小改动、一个失败检查、一个可选依赖缺失、一个有旧授权的会话，观察是否正确复用已有工作、是否误装依赖、是否误报通过。

本次只补充体验与改进建议，没有擅自修改你安装的 skill，也没有声称以上行为试验全部做过。若你决定落实，应先备份、逐条修改、再跑这些场景。

27.1 你的 skill 里面各部分怎么改

部分	当前职责	建议优先补的内容
SKILL.md	什么时候用、不可破坏的规则、怎样交付	简明地要求先确认用户的主要用法和验收方式
workflow.md	分阶段推进；修改会使哪些检查失效	区分已经有证据、需要补证据、明确未验证
coding-practices.md	API、命名空间、依赖与副作用	图形组合的 data/aes/scale/coord 契约
testing-documentation.md	测试、帮助、例子、教程	可复制的人工验收步骤、预期与目的
check-errors.md	定位检查失败	先辨别工具缺失、外部服务故障和包错误
cran-readiness.md	精确制品、跨平台证据、提交准备	不重复询问已明确授权的操作；人工审批单列
scripts/	创建骨架、静态检查、构建检查、skill 验证	工具体检、检查已有 tarball、统一摘要

曾遇到的具体摩擦：skill 验证脚本依赖 PyYAML，但环境没有安装时会先出现 traceback。应在入口给简明依赖提示，帮助命令也应尽量能运行；开发依赖另列清楚。

另一个是环境变量不能只按名字判断：启用时钟检查与禁用时钟检查意义相反，都可能使用同一个变量。应检查生效值，报告“已加强”“已跳过”“未知”，避免错误归类。

skill 修改后，用新包、现有包小改动、已有 tarball、缺可选依赖、缺 TeX、出现 NOTE、已有授权、图形浏览器验收这八类任务试跑，再判断是否改进。未试跑之前，只能称为建议，不能称为已验证更好的版本。

28. 你最终要确认的事情

项目	你需要亲自回答	结果
计数单位	我的一行确实是一个条目，重复行也应计数吗?	待填
指标	三个或更多指标可比较吗？轴序写清了吗?	待填
分组与层级	组均值和分类树是我想表达的东西吗?	待填
分母	默认 root 是否符合读者的理解?	待填
默认图形	灰环已去掉；静态图注是否足够解释指标?	待填
本地安装	新会话从正确包库找到五个函数了吗?	待填
手算	小表均值、计数、比例、范围核对了吗?	待填
静态图	字体、刻度、颜色、图例、导出尺寸可读吗?	待填
交互	真实点过悬停、缩放、组显隐并打开 HTML 吗?	待填
发布证据	当前源码包是否对应当前哈希和检查日志?	待填
署名与维护	姓名邮箱准确，愿意持续处理维护邮件吗?	待填
正式上传	阅读完成后，是否明确同意这份制品提交?	待填

反馈可以写：“第 18 节，两种入口一致，但静态轴名难读；我希望保持无灰环，同时增加别的轴名方案。”给出位置、现象、预期结果，就能直接推进修改。

当前仍是供你审查的版本。不会因为自动检查通过，就替你填写“本人已验收”或上传 CRAN。

29. 想深入时再查这些资料

本书解释以当前仓库代码为准。源码包的精确检查证据看 RELEASE-STATUS.md；新增接口的英文契约看 ?geom_radsun；完整离线教程看 vignette("radar-sunburst", package = "radarsunburst")。

官方资料：

- ggplot2 组合组件与完整图形函数：https://ggplot2-book.org/programming.html
- ggplot2 添加对象的扩展说明：https://ggplot2.tidyverse.org/reference/update_ggplot.html
- ggplot2 主题：https://ggplot2.tidyverse.org/reference/theme.html
- Plotly 转换与图层调整：https://plotly.com/ggplot2/getting-started/
- R 包规范：https://cran.r-project.org/doc/manuals/r-release/R-exts.html
- roxygen2 帮助文档：https://roxygen2.r-lib.org/articles/rd.html
- usethis 创建项目：https://usethis.r-lib.org/reference/create_package.html
- CRAN 政策：https://cran.r-project.org/web/packages/policies.html
- CRAN 提交清单：https://cran.r-project.org/web/packages/submission_checklist.html

你不需要先读完这些资料才能试用。建议先按第 14 到 22 节亲手做一遍，再回看第 12、24、25 节理解开发流程。遇到问题时记录 R 版本、具体命令、错误信息和最小数据，比只说“画不了”更容易定位。